

TASK 2

PRODUCT & CATEGORY MANAGEMENT

SQL Database Management Documentation

1. Objective

The objective of this project is to design and implement a simple Product and Category Management database using SQL. The database stores category and product information, applies database constraints, performs CRUD operations, and generates category-wise analysis.

2. Database Overview

The database is named ProductManagement. It contains two related tables: Category and Product. A category can contain multiple products, creating a one-to-many (1:M) relationship.

3. Category Table Design

Column	Data Type	Constraint
Category_ID	INT	PRIMARY KEY
Category_Name	VARCHAR(50)	NOT NULL, UNIQUE
Description	VARCHAR(200)	Optional

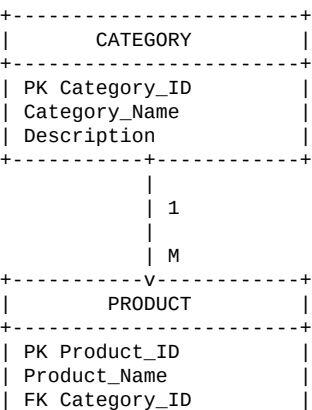
4. Product Table Design

Column	Data Type	Constraint
Product_ID	INT	PRIMARY KEY
Product_Name	VARCHAR(100)	NOT NULL, UNIQUE
Category_ID	INT	NOT NULL, FOREIGN KEY
Price	DECIMAL(10,2)	NOT NULL, CHECK (Price > 0)
Stock_Quantity	INT	NOT NULL, CHECK (Stock_Quantity >= 0)

5. Constraints Used

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each record.
FOREIGN KEY	Connects Product.Category_ID with Category.Category_ID.
NOT NULL	Ensures required fields cannot be empty.
UNIQUE	Prevents duplicate category and product names.
CHECK	Ensures price is greater than 0 and stock quantity is not negative.

6. ER Diagram



Price	
Stock_Quantity	
+-----+	

Relationship: One Category can have many Products (1:M).

7. Sample Category Data

Category_ID	Category_Name	Description
1	Electronics	Electronic devices and accessories
2	Clothing	Men and women clothing
3	Books	Educational and general books
4	Home Appliances	Appliances for home use
5	Sports	Sports equipment and accessories

8. Sample Product Data

Product_ID	Product_Name	Category_ID	Price	Stock
101	Laptop	1	■55,000.00	20
102	Smartphone	1	■25,000.00	35
103	Headphones	1	■2,500.00	50
104	Smart Watch	1	■5,000.00	25
105	T-Shirt	2	■800.00	60
106	Jeans	2	■1,800.00	40
107	Jacket	2	■2,500.00	15
108	Python Programming	3	■900.00	30
109	Database Systems	3	■1,200.00	25
110	Mixer Grinder	4	■3,500.00	18
111	Microwave Oven	4	■8,500.00	10
112	Cricket Bat	5	■4,500.00	12
113	Football	5	■1,200.00	30

9. CRUD Operations

INSERT: Adds a new product record. SELECT: Displays product records. UPDATE: Modifies an existing product. DELETE: Removes a product record.

10. Category-wise Analysis Report

The category-wise analysis groups products according to their categories and calculates product counts, the highest-priced product, categories meeting a specified count condition, and the average product price.

Category	Number of Products
Electronics	4
Clothing	3
Books	2
Home Appliances	2
Sports	2

Highest-priced product: Laptop — ■55,000.00.

Categories having more than 5 products: None in the sample data.

Average product price: ■8,646.15.

11. SQL Queries Used for Analysis

```
SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock_Quantity
FROM Category c
JOIN Product p ON c.Category_ID = p.Category_ID
ORDER BY c.Category_Name;

SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
FROM Category c
LEFT JOIN Product p ON c.Category_ID = p.Category_ID
GROUP BY c.Category_ID, c.Category_Name;

SELECT *
FROM Product
WHERE Price = (SELECT MAX(Price) FROM Product);

SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
```

```
FROM Category c
JOIN Product p ON c.Category_ID = p.Category_ID
GROUP BY c.Category_ID, c.Category_Name
HAVING COUNT(p.Product_ID) > 5;

SELECT AVG(Price) AS Average_Product_Price
FROM Product;
```

12. Query Output Screenshots

After executing the SQL script in MySQL, insert screenshots of the query outputs in this section. Recommended screenshots include table creation, sample data, CRUD operations, category-wise product list, product counts, highest-priced product, HAVING query, and average price.

Screenshot 1 – Table Creation / Sample Data

[Insert SQL output screenshot here]

Screenshot 2 – CRUD Operations

[Insert SQL output screenshot here]

Screenshot 3 – Category-wise Product List

[Insert SQL output screenshot here]

Screenshot 4 – Category Product Count

[Insert SQL output screenshot here]

Screenshot 5 – Highest-priced Product / Average Price

[Insert SQL output screenshot here]

13. Conclusion

The Product and Category Management database was successfully designed using SQL. Primary key, foreign key, NOT NULL, UNIQUE, and CHECK constraints were applied to maintain data integrity. CRUD operations were performed and category-wise queries were used to analyze the product data. The project demonstrates the basic use of relational database design and SQL data management.